```
import { useState, useCallback, useRef, useEffect } from "react"

// ── helpers ──────────────────────────────────────────────────────────────
const fmt = (n) =>
  typeof n === "number"
    ? n.toLocaleString("en-US", { style: "currency", currency: "USD", minimumFrac
    : "$0.00"
const pct = (billed, total) =>
  total > 0 ? Math.min(100, Math.round((billed / total) * 100)) : 0

async function callClaude(messages, system, maxTokens = 4096) {
  const res = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ model: "claude-sonnet-4-20250514", max_tokens: maxToke
  })
  if (!res.ok) {
    const txt = await res.text()
    throw new Error(`API ${res.status}: ${txt.slice(0, 200)}`)
  }
  const data = await res.json()
  if (data.error) throw new Error(`Claude: ${data.error.message}`)
  const text = data.content?.find((b) => b.type === "text")?.text || ""
  if (!text) throw new Error("Empty response from Claude")
  return text
}

function fileToBase64(file) {
  return new Promise((resolve, reject) => {
    const r = new FileReader()
    r.onload = () => resolve(r.result.split(",")[1])
    r.onerror = reject
    r.readAsDataURL(file)
  })
}

// ── Design tokens ────────────────────────────────────────────────────────
const C = {
  border: "0.5px solid var(--color-border-tertiary)",
  borderMid: "0.5px solid var(--color-border-secondary)",
  bg: "var(--color-background-primary)",
  bgSoft: "var(--color-background-secondary)",
  bgPage: "var(--color-background-tertiary)",
  text: "var(--color-text-primary)",
```

```
  muted: "var(--color-text-secondary)",
  hint: "var(--color-text-tertiary)",
  green: "#1D9E75",
  greenDark: "#0F6E56",
  greenLight: "rgba(29,158,117,0.1)",
  amber: "#BA7517",
  amberBg: "rgba(186,117,23,0.1)",
  red: "#A32D2D",
  redBg: "rgba(163,45,45,0.1)",
}

const card = {
  background: C.bg,
  border: C.border,
  borderRadius: 12,
}

const btnBase = {
  fontSize: 12,
  cursor: "pointer",
  fontFamily: "inherit",
  color: C.muted,
  background: "transparent",
  border: C.borderMid,
  borderRadius: 8,
  padding: "5px 11px",
  display: "inline-flex",
  alignItems: "center",
  gap: 5,
}

// ─── Micro components ──────────────────────────────────────────────
function Bar({ value, h = 5 }) {
  const col =
    value >= 100 ? C.green :
    value > 60   ? "#5DCAA5" :
    value > 25   ? "#9FE1CB" : "rgba(128,128,128,0.15)"
  return (
    <div style={{ width: "100%", height: h, background: "rgba(128,128,128,0.08)",
      <div style={{ width: `${Math.min(value, 100)}%`, height: "100%", background
    </div>
  )
}

function Pill({ p }) {
  const [bg, col, label] =
    p >= 100  ? [C.greenLight, C.greenDark, "complete"] :
```

```
    p > 0     ? [C.greenLight, C.green, `${p}%`] :
                ["rgba(136,135,128,0.1)", C.hint, "not started"]
  return (
    <span style={{ fontSize: 11, fontWeight: 500, padding: "2px 9px", borderRadiu
      {label}
    </span>
  )
}

function DropZone({ label, sub, onFile }) {
  const [drag, setDrag] = useState(false)
  const ref = useRef()
  return (
    <div
      onClick={() => ref.current.click()}
      onDragOver={(e) => { e.preventDefault(); setDrag(true) }}
      onDragLeave={() => setDrag(false)}
      onDrop={(e) => { e.preventDefault(); setDrag(false); const f = e.dataTransf
      style={{ border: `1.5px dashed ${drag ? C.green : "rgba(128,128,128,0.2)"}`
    >
      <input ref={ref} type="file" accept=".pdf,.csv,.txt,.doc,.docx,.xlsx,.xls"
      <div style={{ width: 44, height: 44, borderRadius: 12, background: C.greenL
        <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke={C.gre
      </div>
      <p style={{ margin: 0, fontWeight: 500, fontSize: 14, color: C.text }}>{lab
      <p style={{ margin: "5px 0 0", fontSize: 12, color: C.hint }}>{sub}</p>
    </div>
  )
}

function Spinner({ label }) {
  return (
    <div style={{ display: "flex", flexDirection: "column", alignItems: "center",
      <div style={{ width: 32, height: 32, border: "2.5px solid rgba(128,128,128,
      <span style={{ fontSize: 13 }}>{label}</span>
    </div>
  )
}

// ─── SOV Accordion row ──────────────────────────────────────────────
function AreaRow({ room, items, invoiceColumns, isChangeOrder }) {
  const [open, setOpen] = useState(false)
  const total = items.reduce((a, s) => a + (s.scheduledValue || 0), 0)
  const billed = items.reduce((a, s) => a + (s.billedToDate || 0), 0)
  const balance = total - billed
  const p = pct(billed, total)
  const done = p >= 100
```

```
const bgHeader = isChangeOrder ? "rgba(127,119,221,0.05)" : C.bgSoft

return (
  <div style={{ ...card, overflow: "hidden", marginBottom: 6 }}>
    {/* Area header row */}
    <div
      onClick={() => setOpen(!open)}
      style={{ display: "flex", alignItems: "center", gap: 10, padding: "12px 1
    >
      <svg width="14" height="14" viewBox="0 0 16 16" style={{ transform: open
        <path d="M6 4l4 4-4 4" fill="none" stroke={C.hint} strokeWidth="1.5" st
      </svg>
      <div style={{ flex: 1, minWidth: 0 }}>
        <div style={{ display: "flex", justifyContent: "space-between", alignIt
          <span style={{ fontWeight: 500, fontSize: 13, color: done ? C.hint :
            {isChangeOrder && <span style={{ fontSize: 10, background: "rgba(12
            {room}
          </span>
          <div style={{ display: "flex", alignItems: "center", gap: 10, flexShr
            <span style={{ fontSize: 12, color: C.hint, fontVariantNumeric: "ta
            <Pill p={p} />
          </div>
        </div>
        <Bar value={p} />
      </div>
    </div>

    {/* Expanded line items */}
    {open && (
      <div style={{ borderTop: C.border, overflowX: "auto" }}>
        <table style={{ width: "100%", borderCollapse: "collapse", fontSize: 12
          <thead>
            <tr style={{ background: C.bgSoft }}>
              <th style={th("left", 260)}>Description</th>
              <th style={th("right", 50)}>Qty</th>
              <th style={th("right", 50)}>Rate</th>
              <th style={th("right", 90)}>Scheduled</th>
              {invoiceColumns.map((inv) => (
                <th key={inv} style={th("right", 90, true)}>{inv}</th>
              ))}
              <th style={th("right", 90)}>Total billed</th>
              <th style={th("right", 50)}>%</th>
              <th style={th("right", 90)}>Balance</th>
            </tr>
          </thead>
          <tbody>
```

```
              {items.map((s, i) => {
                const itemDone = s.percentComplete >= 100
                return (
                  <tr key={i} style={{ borderTop: C.border, opacity: itemDone ? 0
                    <td style={{ padding: "9px 14px", color: itemDone ? C.hint :
                      {s.description}
                    </td>
                    <td style={{ padding: "9px 8px", textAlign: "right", color: C
                    <td style={{ padding: "9px 8px", textAlign: "right", color: C
                    <td style={{ padding: "9px 8px", textAlign: "right", fontVari
                    {invoiceColumns.map((inv) => {
                      const amount = s.payments?.[inv] || 0
                      return (
                        <td key={inv} style={{ padding: "9px 8px", textAlign: "ri
                          {amount > 0 ? fmt(amount) : "—"}
                        </td>
                      )
                    })}
                    <td style={{ padding: "9px 8px", textAlign: "right", color: s
                    <td style={{ padding: "9px 8px", textAlign: "right" }}><Pill
                    <td style={{ padding: "9px 8px", textAlign: "right", color: i
                  </tr>
                )
              })}
            </tbody>
            {/* Area subtotals */}
            <tfoot>
              <tr style={{ borderTop: `1px solid var(--color-border-secondary)`,
                <td colSpan={4 + invoiceColumns.length} style={{ padding: "8px 14
                <td style={{ padding: "8px 8px", textAlign: "right", fontWeight:
                <td style={{ padding: "8px 8px", textAlign: "right" }}><Pill p={p
                <td style={{ padding: "8px 8px", textAlign: "right", fontVariantN
              </tr>
            </tfoot>
          </table>
        </div>
      )}
    </div>
  )
}

const th = (align, w, inv = false) => ({
  padding: "7px 8px",
  textAlign: align,
  fontWeight: 400,
  color: inv ? C.green : C.hint,
  fontSize: 10,
```

```
  letterSpacing: ".04em",
  textTransform: "uppercase",
  whiteSpace: "nowrap",
  width: w,
  minWidth: w,
  background: inv ? "rgba(29,158,117,0.04)" : undefined,
})

// ── Tabs ──────────────────────────────────────────────────────────────
const TABS = [
  { id: "estimate", label: "Estimate" },
  { id: "sov", label: "Schedule of values" },
  { id: "invoices", label: "Invoices" },
]

// ── Main component ────────────────────────────────────────────────────
export default function FinancialTracker() {
  const [tab, setTab] = useState("estimate")
  const [estimate, setEstimate] = useState(null)   // { projectName, client, tota
  const [sov, setSov] = useState(null)             // { mainContract: [], changeO
  const [invoices, setInvoices] = useState([])
  const [loading, setLoading] = useState("")
  const [error, setError] = useState("")
  const [projectName, setProjectName] = useState("Untitled project")
  const [client, setClient] = useState("")
  const [contractDate, setContractDate] = useState("")
  const [editingName, setEditingName] = useState(false)
  const nameRef = useRef()

  useEffect(() => { if (editingName && nameRef.current) nameRef.current.focus() }

  // Derived totals
  const allItems = sov ? [...(sov.mainContract || []), ...(sov.changeOrders || []
  const totalScheduled = allItems.reduce((a, s) => a + (s.scheduledValue || 0), 0
  const totalBilled = allItems.reduce((a, s) => a + (s.billedToDate || 0), 0)
  const totalBalance = totalScheduled - totalBilled
  const overallPct = pct(totalBilled, totalScheduled)
  const invoiceColumns = sov?.invoiceColumns || []

  // ── Parse estimate ──────────────────────────────────────────────────
  const parseEstimate = useCallback(async (file) => {
    setLoading("Parsing estimate with AI…")
    setError("")
    try {
      const base64 = await fileToBase64(file)
      const mt = file.type || "application/pdf"
      const isPdf = mt === "application/pdf"
```

```
const isImg = mt.startsWith("image/")
const prompt = [
  "Extract all line items from this construction estimate.",
  "Return ONLY a raw JSON object — no markdown, no backticks, no explanatio
  "Schema: projectName (string), client (string), estimateNumber (string),
  "areas (array of: name string, floor string, lineItems array of: descript
  "Use room/area section headers (FOYER, KITCHEN, MASTER BATHROOM, etc.) as
  "Use the parent floor label (FIRST FLOOR, SECOND FLOOR, ADU, EXTERIOR) as
  "For items with $0 amount marked REMOVED: include them with isRemoved=tru
  "Include ALL line items that have a dollar amount > 0.",
  "Return raw JSON only. Start your response with { and end with }.",
].join(" ")

let messages
if (isPdf || isImg) {
  messages = [{ role: "user", content: [
    { type: isPdf ? "document" : "image", source: { type: "base64", media_t
    { type: "text", text: prompt }
  ]}]
} else {
  const text = await file.text()
  messages = [{ role: "user", content: `${prompt}\n\nESTIMATE:\n${text.slic
}

const raw = await callClaude(messages, "Construction estimating assistant.
let cleaned = raw.replace(/```json|```/g, "").trim()
// attempt to close truncated JSON
if (!cleaned.endsWith("}")) {
  const lastBrace = cleaned.lastIndexOf("}")
  if (lastBrace > 0) cleaned = cleaned.slice(0, lastBrace + 1)
}
const parsed = JSON.parse(cleaned)
setEstimate(parsed)
if (parsed.projectName) setProjectName(parsed.projectName)
if (parsed.client) setClient(parsed.client)

// Build initial SOV structure from estimate
let itemNum = 1
const mainContract = []
for (const area of parsed.areas || []) {
  for (const li of area.lineItems || []) {
    if (li.isRemoved && li.amount === 0) continue
    mainContract.push({
      id: itemNum++,
      room: area.name,
      floor: area.floor,
      description: li.description,
```

```
            qty: li.qty,
            rate: li.rate,
            scheduledValue: li.amount || 0,
            billedToDate: 0,
            percentComplete: 0,
            balanceToFinish: li.amount || 0,
            payments: {},
            invoiceRefs: [],
            isRemoved: li.isRemoved || false,
          })
        }
      }

      setSov({ mainContract, changeOrders: [], invoiceColumns: [] })
      setTab("sov")
    } catch (e) {
      console.error("Estimate parse error:", e)
      setError(`Parse failed: ${e.message || "Unknown error"}. Check the browser
    } finally {
      setLoading("")
    }
  }, [])

  // ── Apply invoice ──────────────────────────────────────────────────────
  const parseInvoice = useCallback(async (file) => {
    if (!sov || !sov.mainContract.length) { setError("Upload an estimate first.")
    setLoading("Matching invoice to SOV…")
    setError("")
    try {
      const base64 = await fileToBase64(file)
      const mt = file.type || "application/pdf"
      const isPdf = mt === "application/pdf"
      const isImg = mt.startsWith("image/")

      const sovSummary = [...sov.mainContract, ...sov.changeOrders]
        .map((s) => `ID ${s.id}: [${s.room}] ${s.description} (scheduled: $${s.sc
        .join("\n")

      const prompt = `This is a construction invoice or draw request. Match its l
Return ONLY valid JSON:
{
  "invoiceNumber": "string (e.g. Inv 4598 or Draw 1)",
  "invoiceDate": "string",
  "invoiceTotal": number,
  "matches": [
    { "sovId": number, "amountBilled": number }
  ]
```

```
}
Rules: Only match items explicitly in the invoice. amountBilled cannot exceed sch

SOV ITEMS:
${sovSummary}`

      let messages
      if (isPdf || isImg) {
        messages = [{ role: "user", content: [
          { type: isPdf ? "document" : "image", source: { type: "base64", media_t
          { type: "text", text: prompt }
        ]}]
      } else {
        const text = await file.text()
        messages = [{ role: "user", content: `${prompt}\n\nINVOICE:\n${text.slice
      }

      const raw = await callClaude(messages, "Construction billing assistant. Ret
      const inv = JSON.parse(raw.replace(/```json|```/g, "").trim())
      const invLabel = inv.invoiceNumber || `Invoice ${invoices.length + 1}`

      const applyPayments = (items) =>
        items.map((item) => {
          const match = inv.matches?.find((m) => m.sovId === item.id)
          if (!match || !match.amountBilled) return item
          const newBilled = Math.min(item.billedToDate + match.amountBilled, item
          return {
            ...item,
            billedToDate: newBilled,
            percentComplete: pct(newBilled, item.scheduledValue),
            balanceToFinish: item.scheduledValue - newBilled,
            payments: { ...item.payments, [invLabel]: match.amountBilled },
            invoiceRefs: [...item.invoiceRefs, invLabel],
          }
        })

      setSov((prev) => ({
        ...prev,
        mainContract: applyPayments(prev.mainContract),
        changeOrders: applyPayments(prev.changeOrders),
        invoiceColumns: [...(prev.invoiceColumns || []), invLabel],
      }))
      setInvoices((prev) => [...prev, { ...inv, invoiceNumber: invLabel, fileName
      setTab("sov")
    } catch (e) {
      console.error("Invoice parse error:", e)
      setError(`Invoice parse failed: ${e.message || "Unknown error"}`)
```

```
  } finally {
    setLoading("")
  }
}, [sov, invoices])

// ── Export SOV as CSV ──────────────────────────────────────────────
const exportCSV = () => {
  if (!sov) return
  const allRows = [...sov.mainContract, ...sov.changeOrders]
  const header = ["Room", "Description", "Qty", "Rate", "Scheduled Value", ...i
  const rows = allRows.map((s) => [
    s.room, s.description, s.qty, s.rate, s.scheduledValue,
    ...invoiceColumns.map((inv) => s.payments?.[inv] || 0),
    s.billedToDate, s.percentComplete + "%", s.balanceToFinish,
  ])
  const totRow = [
    "TOTAL", "", "", "", totalScheduled,
    ...invoiceColumns.map((inv) => allRows.reduce((a, s) => a + (s.payments?.[i
    totalBilled, overallPct + "%", totalBalance,
  ]
  const csv = [header, ...rows, [], totRow].map((r) => r.map((c) => `"${c ?? ""
  const a = Object.assign(document.createElement("a"), {
    href: URL.createObjectURL(new Blob([csv], { type: "text/csv" })),
    download: `${projectName.replace(/\s+/g, "_")}_SOV.csv`,
  })
  a.click()
}

// ── Group SOV items by room ────────────────────────────────────────
const groupByRoom = (items) => {
  const map = {}
  for (const item of items) {
    if (!map[item.room]) map[item.room] = []
    map[item.room].push(item)
  }
  return map
}

const mainGroups = sov ? groupByRoom(sov.mainContract) : {}
const coGroups = sov ? groupByRoom(sov.changeOrders) : {}
const changeOrderTotal = (sov?.changeOrders || []).reduce((a, s) => a + (s.sche
const changeOrderBilled = (sov?.changeOrders || []).reduce((a, s) => a + (s.bil

return (
  <div style={{ display: "grid", gridTemplateColumns: "210px 1fr", height: 680,

    {/* ── Sidebar ── */}
```

```
<div style={{ display: "flex", flexDirection: "column", background: C.bg, b

  {/* Brand */}
  <div style={{ padding: "17px 16px 14px", borderBottom: C.border, display:
    <div style={{ width: 30, height: 30, borderRadius: 8, background: "line
      <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="#
    </div>
    <div>
      <div style={{ fontSize: 13, fontWeight: 500, color: C.text }}>Financi
      <div style={{ fontSize: 10, color: C.hint }}>Progress billing</div>
    </div>
  </div>

  {/* Project meta */}
  <div style={{ padding: "13px 14px 12px", borderBottom: C.border }}>
    <p style={{ fontSize: 10, color: C.hint, textTransform: "uppercase", le
    {editingName ? (
      <input
        ref={nameRef}
        value={projectName}
        onChange={(e) => setProjectName(e.target.value)}
        onBlur={() => setEditingName(false)}
        onKeyDown={(e) => e.key === "Enter" && setEditingName(false)}
        style={{ width: "100%", fontSize: 13, fontWeight: 500, background:
      />
    ) : (
      <div onClick={() => setEditingName(true)} title="Click to rename" sty
        {projectName}
      </div>
    )}
    {client && <p style={{ margin: "4px 5px 0", fontSize: 11, color: C.hint
    <div style={{ marginTop: 10 }}>
      <p style={{ fontSize: 10, color: C.hint, textTransform: "uppercase",
      <input type="date" value={contractDate} onChange={(e) => setContractD
    </div>
  </div>

  {/* Nav */}
  <nav style={{ padding: "10px 8px" }}>
    <p style={{ fontSize: 10, color: C.hint, textTransform: "uppercase", le
    {TABS.map((t) => {
      const active = tab === t.id
      return (
        <button key={t.id} onClick={() => setTab(t.id)} style={{
          display: "flex", alignItems: "center", gap: 8, width: "100%", pad
          borderRadius: 9, border: "none", cursor: "pointer", fontFamily: "
          background: active ? C.greenLight : "transparent",
```

```
            color: active ? C.greenDark : C.muted,
            fontWeight: active ? 500 : 400, fontSize: 13, textAlign: "left",
            marginBottom: 2, transition: "all .12s",
          }}>
            {t.id === "invoices" && invoices.length > 0 && (
              <span style={{ marginLeft: "auto", fontSize: 10, background: C.
            )}
            {t.label}
          </button>
        )
      })}
    </nav>

    {/* Live summary footer */}
    {sov && (
      <div style={{ marginTop: "auto", padding: "12px 14px", borderTop: C.bor
        <div style={{ display: "flex", justifyContent: "space-between", margi
          <span style={{ fontSize: 11, color: C.hint }}>Overall progress</spa
          <span style={{ fontSize: 11, fontWeight: 500, color: overallPct >=
        </div>
        <Bar value={overallPct} />
        <div style={{ marginTop: 9, display: "flex", flexDirection: "column",
          {[
            { label: "Contract", val: fmt((estimate?.totalAmount || 0)) },
            { label: "Billed", val: fmt(totalBilled), accent: true },
            { label: "Balance", val: fmt(totalBalance) },
            { label: "Invoices", val: String(invoices.length) },
          ].map((c) => (
            <div key={c.label} style={{ display: "flex", justifyContent: "spa
              <span style={{ fontSize: 10, color: C.hint }}>{c.label}</span>
              <span style={{ fontSize: 11, fontWeight: 500, color: c.accent ?
            </div>
          ))}
        </div>
      </div>
    )}
  </div>

  {/* ── Main panel ── */}
  <div style={{ display: "flex", flexDirection: "column", overflow: "hidden"

    {/* Top bar */}
    <div style={{ display: "flex", alignItems: "center", justifyContent: "spa
      <div>
        <p style={{ margin: 0, fontSize: 15, fontWeight: 500, color: C.text }
          {tab === "estimate" ? "Estimate" : tab === "sov" ? "Schedule of val
        </p>
```

```
    <p style={{ margin: "1px 0 0", fontSize: 11, color: C.hint }}>
      {sov
        ? `${sov.mainContract.length} main items · ${sov.changeOrders.len
        : "No estimate loaded"}
    </p>
  </div>
  {sov && (
    <div style={{ display: "flex", alignItems: "center", gap: 14 }}>
      <div style={{ textAlign: "right" }}>
        <p style={{ margin: 0, fontSize: 22, fontWeight: 500, color: over
        <p style={{ margin: 0, fontSize: 10, color: C.hint }}>billed to d
      </div>
      {/* Radial ring */}
      <div style={{ width: 46, height: 46, position: "relative", flexShri
        <svg width="46" height="46" viewBox="0 0 46 46" style={{ position
          <circle cx="23" cy="23" r="19" fill="none" stroke="rgba(128,128
          <circle cx="23" cy="23" r="19" fill="none" stroke={overallPct >
            strokeDasharray={`${2 * Math.PI * 19}`}
            strokeDashoffset={`${2 * Math.PI * 19 * (1 - overallPct / 100
            strokeLinecap="round"
            style={{ transition: "stroke-dashoffset .6s ease" }} />
        </svg>
      </div>
    </div>
  )}
</div>

{/* Hero progress bar + key metrics */}
{sov && (
  <div style={{ padding: "12px 20px 13px", background: C.bg, borderBottom
    <Bar value={overallPct} h={8} />
    <div style={{ display: "grid", gridTemplateColumns: "repeat(5, 1fr)",
      {[
        { label: "Main contract", val: fmt(estimate?.totalAmount || 0) },
        { label: "Change orders", val: fmt(changeOrderTotal) },
        { label: "Billed to date", val: fmt(totalBilled), accent: true },
        { label: "Balance", val: fmt(totalBalance) },
        { label: "Applications", val: String(invoices.length) },
      ].map((c, i) => (
        <div key={c.label} style={{ textAlign: "center", borderLeft: i >
          <p style={{ margin: 0, fontSize: 10, color: C.hint, textTransfo
          <p style={{ margin: "3px 0 0", fontSize: 13, fontWeight: 500, c
        </div>
      ))}
    </div>
  </div>
)}
```

```
{/* Error */}
{error && (
  <div style={{ margin: "10px 20px 0", padding: "9px 13px", background: "
    <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="c
    {error}
    <button onClick={() => setError("")} style={{ marginLeft: "auto", bac
  </div>
)}

{/* Content */}
<div style={{ flex: 1, overflow: "auto", padding: "14px 20px" }}>
  {loading ? <Spinner label={loading} /> : null}

  {/* ── ESTIMATE TAB ── */}
  {!loading && tab === "estimate" && (
    !estimate ? (
      <DropZone
        label="Upload your estimate"
        sub="PDF, image, CSV, or text — AI extracts every area and line i
        onFile={parseEstimate}
      />
    ) : (
      <div>
        <div style={{ display: "flex", justifyContent: "space-between", a
          <div>
            <p style={{ margin: 0, fontWeight: 500, fontSize: 14, color:
              Estimate #{estimate.estimateNumber} · {estimate.date}
            </p>
            <p style={{ margin: "2px 0 0", fontSize: 12, color: C.hint }}
              {estimate.areas?.length} areas · {estimate.areas?.reduce((a
            </p>
          </div>
          <div style={{ display: "flex", gap: 8 }}>
            <button onClick={() => { setEstimate(null); setSov(null); set
          </div>
        </div>
        {estimate.areas?.map((area, i) => (
          <div key={i} style={{ ...card, marginBottom: 10, overflow: "hid
            <div style={{ display: "flex", justifyContent: "space-between
              <span style={{ fontWeight: 500, fontSize: 13, color: C.text
              <span style={{ fontSize: 13, color: C.muted, fontVariantNum
                {fmt(area.lineItems?.reduce((a, li) => a + (li.amount ||
              </span>
            </div>
            <table style={{ width: "100%", borderCollapse: "collapse", fo
              <thead>
```

```
                <tr>
                  {["Description", "Qty", "Rate", "Amount"].map((h) => (
                    <th key={h} style={th(h === "Description" ? "left" :
                  ))}
                </tr>
              </thead>
              <tbody>
                {area.lineItems?.map((li, j) => (
                  <tr key={j} style={{ borderTop: C.border, opacity: li.i
                    <td style={{ padding: "8px 14px", color: li.isRemoved
                    <td style={{ padding: "8px 8px", textAlign: "right",
                    <td style={{ padding: "8px 8px", textAlign: "right",
                    <td style={{ padding: "8px 8px", textAlign: "right",
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        ))}
        <div style={{ display: "flex", justifyContent: "flex-end", paddin
          Total: {fmt(estimate.totalAmount)}
        </div>
      </div>
    )
  )}

  {/* ── SOV TAB ── */}
  {!loading && tab === "sov" && (
    !sov ? (
      <div style={{ textAlign: "center", padding: "4rem", color: C.hint,
        Upload an estimate to auto-generate the schedule of values.
      </div>
    ) : (
      <div>
        <div style={{ display: "flex", justifyContent: "space-between", a
          <p style={{ margin: 0, fontSize: 12, color: C.hint }}>
            Click any area to expand line items {invoiceColumns.length >
          </p>
          <div style={{ display: "flex", gap: 8 }}>
            <button onClick={exportCSV} style={btnBase}>
              <svg width="13" height="13" viewBox="0 0 24 24" fill="none"
              Export CSV
            </button>
          </div>
        </div>

        {/* Main contract */}
```

```
      {Object.keys(mainGroups).length > 0 && (
        <div style={{ marginBottom: 20 }}>
          <div style={{ display: "flex", alignItems: "center", gap: 8,
            <span style={{ fontSize: 11, fontWeight: 500, color: C.hint
            <div style={{ flex: 1, height: "0.5px", background: "var(--
            <span style={{ fontSize: 11, color: C.hint, fontVariantNume
          </div>
          {Object.entries(mainGroups).map(([room, items]) => (
            <AreaRow key={room} room={room} items={items} invoiceColumn
          ))}
        </div>
      )}

      {/* Change orders */}
      {Object.keys(coGroups).length > 0 && (
        <div>
          <div style={{ display: "flex", alignItems: "center", gap: 8,
            <span style={{ fontSize: 11, fontWeight: 500, color: C.hint
            <div style={{ flex: 1, height: "0.5px", background: "var(--
            <span style={{ fontSize: 11, color: C.hint, fontVariantNume
          </div>
          {Object.entries(coGroups).map(([room, items]) => (
            <AreaRow key={room} room={room} items={items} invoiceColumn
          ))}
        </div>
      )}

      {/* Grand total bar */}
      <div style={{ ...card, padding: "13px 18px", marginTop: 16, displ
        <span style={{ fontWeight: 500, fontSize: 13 }}>Grand total</sp
        <div style={{ display: "flex", gap: 24, fontVariantNumeric: "ta
          <span style={{ color: C.hint }}>Scheduled: {fmt(totalSchedule
          <span style={{ color: C.greenDark, fontWeight: 500 }}>Billed:
          <span style={{ color: C.text }}>Balance: {fmt(totalBalance)}<
          <span style={{ color: C.greenDark, fontWeight: 500 }}>{overal
        </div>
      </div>
    </div>
  )
)}

{/* ── INVOICES TAB ── */}
{!loading && tab === "invoices" && (
  <div style={{ display: "flex", flexDirection: "column", gap: 16 }}>
    {!sov ? (
      <p style={{ color: C.hint, fontSize: 13, textAlign: "center", pad
    ) : (
```

```
              <DropZone label="Upload a payment application / invoice" sub="AI
            )}
            {invoices.length > 0 && (
              <div>
                <p style={{ fontWeight: 500, fontSize: 13, margin: "0 0 10px",
                {invoices.map((inv, i) => (
                  <div key={i} style={{ ...card, padding: "13px 16px", marginBo
                    <div style={{ display: "flex", alignItems: "center", gap: 1
                      <div style={{ width: 34, height: 34, borderRadius: 9, bac
                        <svg width="16" height="16" viewBox="0 0 24 24" fill="n
                      </div>
                      <div>
                        <p style={{ margin: 0, fontWeight: 500, fontSize: 13, c
                        <p style={{ margin: "2px 0 0", fontSize: 11, color: C.h
                          {inv.invoiceDate ? `Date: ${inv.invoiceDate} · ` : ""
                        </p>
                      </div>
                    </div>
                    <div style={{ textAlign: "right" }}>
                      <p style={{ margin: 0, fontWeight: 500, fontSize: 14, col
                      <p style={{ margin: "2px 0 0", fontSize: 11, color: C.hin
                    </div>
                  </div>
                ))}
              </div>
            )}
          </div>
        )}
      </div>
    </div>

    <style>{`@keyframes spin{to{transform:rotate(360deg)}}`}</style>
  </div>
 )
}
```